# Machine Intelligence

## 4. Classical Planning: Representation and Relaxation

Daniel Fišer

AALBORG UNIVERSITY

Fall 2025

# Agenda


1. Introduction
2. Finite Domain Representation
3. Delete Relaxation Heuristics
4. FF Heuristic

# Planning

**Ambition:**

**Write one program (planner) that can solve all sequential decision-making problems.**

**How do we describe our problem to the planner?**

- A *logical description* of the possible states
- A *logical description* of the initial state $I$
- A *logical description* of the goal condition $G$
- *logical description* of the set $\mathcal{A}$ of actions in terms of preconditions and effects

$\rightsquigarrow$ Solution (plan) = sequence of actions from $\mathcal{A}$, transforming $I$ into a state that satisfies $G$.

# Algorithmic Problems in Planning

## Satisficing Planning

**Input:** A planning task $\Pi$.
**Output:** A plan for $\Pi$, or "unsolvable" if no plan for $\Pi$ exists.

## Optimal Planning

**Input:** A planning task $\Pi$.
**Output:** An *optimal* plan for $\Pi$, or "unsolvable" if no plan for $\Pi$ exists.

$\rightsquigarrow$ The techniques successful for either one of these are almost disjoint. And satisficing planning is *much* more effective in practice.

$\rightsquigarrow$ Programs solving these problems are called (optimal) planners, planning systems, or planning tools.

# Our Agenda for This Chapter

- **The FDR Planning Formalism:** How do we represent a planning task?
  $\rightsquigarrow$ Lays the framework we'll be looking at.
- **Delete Relaxation:** How to relax a planning problem?
  $\rightsquigarrow$ The delete relaxation is the most successful method for the *automatic* generation of heuristic functions. It relaxes FDR planning tasks by ignoring "delete" effects.
  $\rightsquigarrow$ $h^+$ is the "ideal" delete relaxation heuristic.
- **FF Heuristic**: How to approximate $h^+$?
  $\rightsquigarrow$ Turns out that, in practice, we must approximate $h^+$.

# FDR: Syntax

## Definition (FDR Planning Task)

A finite-domain representation planning task, short FDR planning task, is a 5-tuple $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ where:

- $\mathcal{V}$ is a finite set of state variables, each $v \in \mathcal{V}$ with a finite domain $D_v$.

  We refer to (partial) functions on $\mathcal{V}$, mapping each $v \in \mathcal{V}$ into a member of $D_v$, as (partial) states.
- $\mathcal{A}$ is a finite set of actions; each $a \in \mathcal{A}$ is a pair $(\mathrm{pre}_a, \mathrm{eff}_a)$ of partial states referred to as the action's preconditions and effects; we require that $\mathrm{pre}_a(v) \neq \mathrm{eff}_a(v)$ whenever both are defined.
- $\mathrm{cost} : \mathcal{A} \mapsto \mathbb{N}_0$ is the cost function.
- $I$ is the initial state.
- $G$ is a partial state called the goal.

# FDR: Additional Notation

- Pairs $(v, d)$, where $v \in \mathcal{V}$ and $d \in D_v$, are called facts, also written as $v = d$.
- We define $\mathcal{V}(p) := \{v \in V \mid p(v) \text{ is defined}\}$ for partial states $p$, i.e., $\mathcal{V}(p)$ is the set of variables on which $p$ is defined.
- We identify partial states $p$ with fact sets, i.e., we can write $p = \{(v, d) \mid v \in \mathcal{V}(p), p(v) = d\}$.

  This allows us to write, e.g., $p \subseteq s$ to say that $s$ agrees with $p$ on all $v \in \mathcal{V}(p)$. In other words, $p \subseteq s$ holds if and only if $p(v) = s(v)$ for all variables on which $p$ is defined.

# FDR: Semantics

## Definition (FDR State Space)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task. The state space of $\Pi$ is the tuple $\Theta_\Pi = (\mathcal{S}, \mathcal{A}, \mathrm{cost}, \mathcal{T}, I, \mathcal{S}_G)$ where:

- $\mathcal{S}$ is the set of all states over $\mathcal{V}$.
- $\mathcal{T} = \{s \xrightarrow{a} s' \mid s, s' \in \mathcal{S}, a \in \mathcal{A}(s), s' = s[\![a]\!]\}$, where $\mathcal{A}(s) := \{a \in \mathcal{A} \mid \mathrm{pre}_a \subseteq s\}$ and for all $a \in \mathcal{A}(s)$
$$s[\![a]\!](v) := \begin{cases} \mathrm{eff}_a(v) & \text{if } v \in \mathcal{V}(\mathrm{eff}_a) \\ s(v) & \text{if } v \notin \mathcal{V}(\mathrm{eff}_a) \end{cases}$$
- $\mathcal{S}_G = \{s \in \mathcal{S} \mid G \subseteq s\}$.

$\rightsquigarrow$ $\mathcal{A}(s)$ is the set of all actions applicable in $s$.
$\rightsquigarrow$ $s[\![a]\!]$ describes how we apply action $a$ on state $s$: We rewrite values of all variables defined in $\mathrm{eff}_a$ according to $\mathrm{eff}_a$.

# FDR: Semantics

## Definition (FDR State Space)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task. The state space of $\Pi$ is the tuple $\Theta_\Pi = (\mathcal{S}, \mathcal{A}, \mathrm{cost}, \mathcal{T}, I, \mathcal{S}_G)$ where:

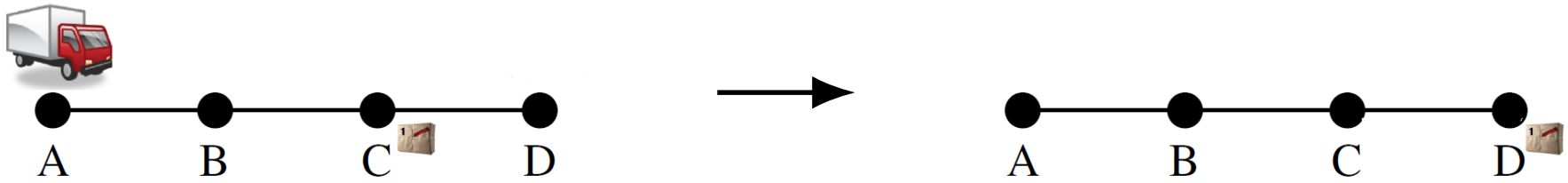
- $\mathcal{S}$ is the set of all states over $\mathcal{V}$.
- $\mathcal{T} = \{s \xrightarrow{a} s' \mid s, s' \in \mathcal{S}, a \in \mathcal{A}(s), s' = s[\![a]\!]\}$, where $\mathcal{A}(s) := \{a \in \mathcal{A} \mid \mathrm{pre}_a \subseteq s\}$ and for all $a \in \mathcal{A}(s)$
$$s[\![a]\!](v) := \begin{cases} \mathrm{eff}_a(v) & \text{if } v \in \mathcal{V}(\mathrm{eff}_a) \\ s(v) & \text{if } v \notin \mathcal{V}(\mathrm{eff}_a) \end{cases}$$
- $\mathcal{S}_G = \{s \in \mathcal{S} \mid G \subseteq s\}$.

## Definition (Plans)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task, and $\Theta_\Pi$ its state space. A sequence of actions $\pi = (a_1, \dots, a_n) \in \mathcal{A}^n$ is called $s$-plan if there is a path $s \xrightarrow{a_1} \dots \xrightarrow{a_n} s_g$ in $\Theta_\Pi$ such that $s_g$ is a goal state. $I$-plan is called simply plan. The cost of $\pi$ is defined as $\mathrm{cost}(\pi) = \sum_{i \in \{1,\dots,n\}} \mathrm{cost}(a_i)$.
An $s$-plan $\pi$ is called optimal if $\mathrm{cost}(\pi)$ is minimal among all $s$-plans.

$\rightsquigarrow$ We also say "plan for $\Pi$" or "plan in $\Theta_\Pi$".

# FDR Encoding of Simple Logistics Problem




Variables $\mathcal{V} = \{truck, pkg\}$,

- domains $D_{truck} = \{A, B, C, D\}$, $D_{pkg} = \{A, B, C, D, t\}$.

Initial state $I = \{truck = A, pkg = C\}$

Goal $G = \{pkg = D\}$

Actions $a \in \mathcal{A}$:

$drive(x, y)$ for all $(x, y) \in \{(A, B), (B, A), (B, C), (C, B), (C, D), (D, C)\}$

- $\text{pre}_{drive(x,y)} = \{truck = x\}$, $\text{eff}_{drive(x,y)} = \{truck = y\}$

$load(x)$ for all $x \in \{A, B, C, D\}$

- $\text{pre}_{load(x)} = \{truck = x, package = x\}$, $\text{eff}_{load(x)} = \{package = t\}$

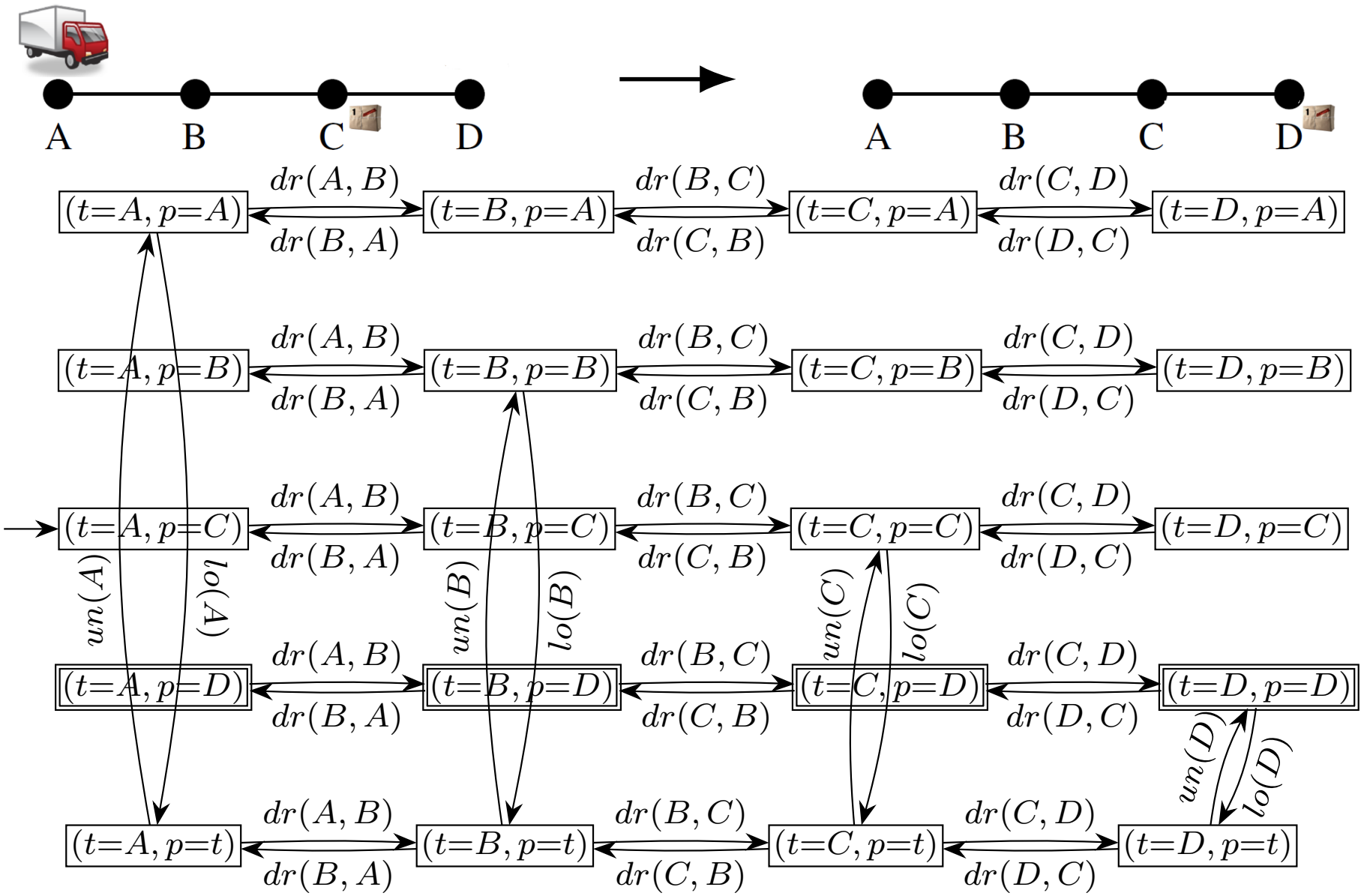
$unload(x)$ for all $x \in \{A, B, C, D\}$

- $\text{pre}_{unload(x)} = \{truck = x, package = t\}$, $\text{eff}_{unload(x)} = \{package = x\}$

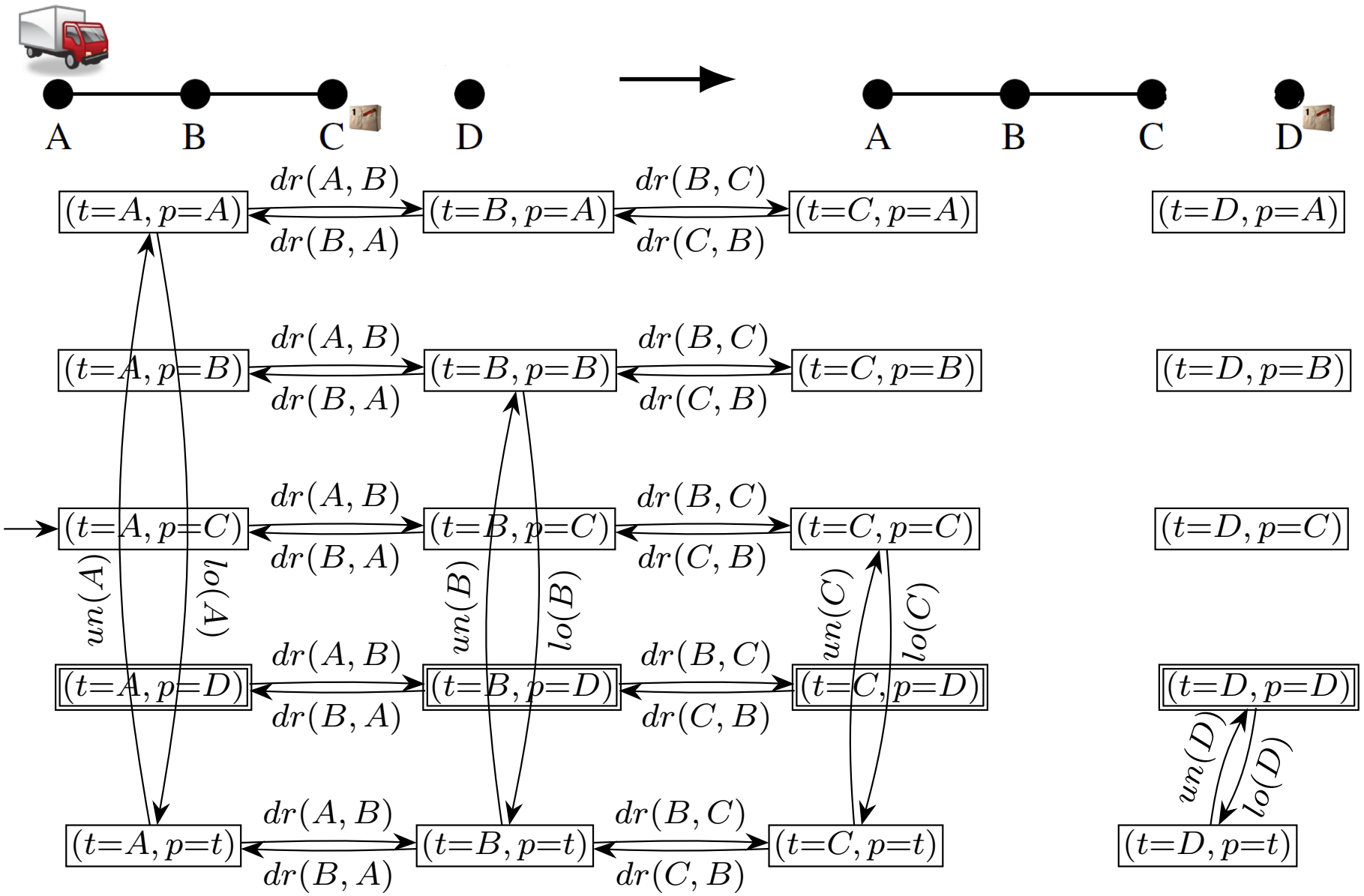
Cost function $\text{cost}(a) = 1$ for all $a \in \mathcal{A}$

A plan: $(drive(A, B), drive(B, C), load(C), drive(C, D), unload(D))$

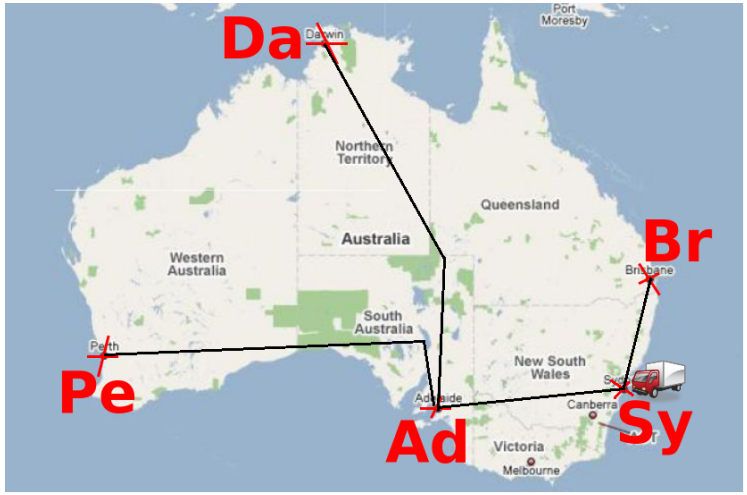
# FDR Simple Logistics Problem: State Space



# FDR Unsolvable Logistics Problem: State Space



# FDR Encoding of Australia Traveling Salesman




- Variables $\mathcal{V}$: $at : \{Sydney, Adelaide, Brisbane, Perth, Darwin\}$; $visited(x) : \{T, F\}$ for $x \in \{Sydney, Adelaide, Brisbane, Perth, Darwin\}$.
- Initial state $I$: $at = Sydney, visited(Sydney) = T, visited(x) = F$ for $x \neq Sydney$.
- Goal $G$: $at = Sydney, visited(x) = T$ for all $x$.
- Actions $a \in \mathcal{A}$: $drive(x, y)$ where $x, y$ have a road.
  - Precondition $\text{pre}_a$: $at = x$.
  - Effect $\text{eff}_a$: $at = y, visited(y) = T$.
- Cost function cost:

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sydney, Brisbane\} \\ 2 & \{x, y\} = \{Sydney, Adelaide\} \\ 3 & \{x, y\} = \{Adelaide, Perth\} \\ 4 & \{x, y\} = \{Adelaide, Darwin\} \end{cases}$$

- A plan: $(drive(Sy, Br), drive(Br, Sy), drive(Sy, Ad), ...)$

# Blackbox vs. Declarative Representations

Remember this from Lecture 2?

Declarative representations are strictly more powerful than blackbox ones. They allow to implement the API, and much more (e.g., analyze/simplify the task).

**Blackbox Description of an FDR planning task $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$**

- InitialState(): `return` $I$
- GoalTest($s$): `return` $G \subseteq s$
- Cost($a$): `return` $\mathrm{cost}(a)$
- Actions($s$): `return` $\mathcal{A}(s)$
- Successor($s, a$): `return` $s[\![a]\!]$

So, we can immediatelly apply everything we already know about state spaces and search: We can solve planning tasks using a state space search algorithms.

**But:** Now we can construct heuristic functions in domain-independent manner! That is, we do not need to think about each particular planning problem. **And** we can apply additional automated techniques, e.g., for pruning of state spaces, or planning tasks. (And study sequential decision making in general.)

# Few Notes on Computational Complexity

**Theorem (Plan Existence)**

Given an FDR planning task $\Pi$, deciding whether there exists a plan for $\Pi$ is PSPACE-complete.

**Theorem (Optimal Plan Existence)**

Given an FDR planning task $\Pi$ and an integer $K$, deciding whether there exists a plan for $\Pi$ of length at most $K$ is PSPACE-complete.

**Theorem (Polynomially Bounded Optimal Plan Existence)**

Given an FDR planning task $\Pi$ and an integer $K$ bounded by a polynomial in size of $\Pi$, deciding whether there exists a plan for $\Pi$ of length at most $K$ is NP-complete.

**Key takeaways:**

- Planning in general is **hard**: unless NP = P and PSPACE = NP, there is no efficient (polynomial) algorithm solving *any* planning task.
- Plans can be exponentially long in the size of $\Pi$. But even polynomially bounded planning is **hard** (unless NP = P).

# Exercise Sheet

Exercise Sheet 3: Section 1

# Reminder about Heuristics

## Definition (Heuristic Function)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \text{cost}, I, G)$ be an FDR planning task with state space $\Theta_\Pi$ and states $\mathcal{S}$. A heuristic (function) for $\Pi$ is a function $h : \mathcal{S} \mapsto \mathbb{N}_0 \cup \{\infty\}$ such that $h(s_g) = 0$ for every goal state $s_g$.

The optimal heuristic, denoted as $h^*$, is the heuristic function assigning every $s \in \mathcal{S}$ the cost of an optimal $s$-plan, or $\infty$ if there is no $s$-plan (in which case we say that $s$ is a dead-end).

A heuristic $h$ for $\Pi$ is called:

- safe if $h^*(s) = \infty$ for all states $s \in \mathcal{S}$ such that $h(s) = \infty$,
- admissible if $h(s) \leq h^*(s)$ for all states $s \in \mathcal{S}$,
- consistent if $h(s) - h(s') \leq \text{cost}(a)$ for all transitions $s \xrightarrow{a} s'$ in $\Theta_\Pi$.

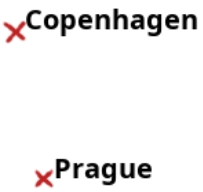
## Question

Fill the blank: $h^*(I)$ is the cost of _____.

A: $h^*(I)$ is the cost of **optimal plans**.

# Reminder about Heuristics: How to Relax?

## Do you remember how to relax?



| | | | |
|---|---|---|---|
| 9 | 2 | 12 | 6 |
| 5 | 7 | 14 | 13 |
| 3 | 4 | 1 | 11 |
| 15 | 10 | 8 | |

—

| | | | |
|---|---|---|---|
| | 2 | | 6 |
| 5 | 7 | | |
| 3 | 4 | 1 | |
| | | | |

# How the Delete Relaxation Changes the World

Delete relaxation:

**"When the world changes, its previous state remains true as well."**

Relaxed world: (before)

# How the Delete Relaxation Changes the World

Delete relaxation:

**"When the world changes, its previous state remains true as well."**

Relaxed world: (after)

# Delete Relaxation in FDR

## Definition (FDR Delete-Relaxed State Space)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task. The delete-relaxed state space of $\Pi$ is the tuple $\Theta^+_\Pi = (\mathcal{S}^+, \mathcal{A}, \mathrm{cost}, \mathcal{T}^+, I, \mathcal{S}^+_G)$ where:

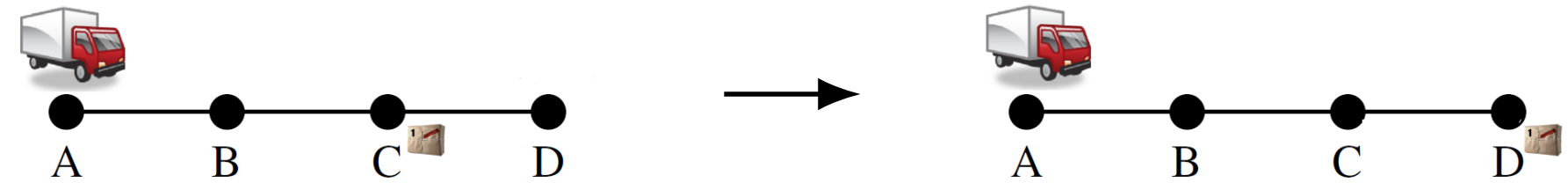
- $\mathcal{S}^+ = \mathcal{P}(\mathcal{F})$, where $\mathcal{P}(\cdot)$ denotes power set, and $\mathcal{F}$ is the set of all facts, i.e., $\mathcal{F} = \{(v, d) \mid v \in \mathcal{V}, d \in D_v\}$.
- $\mathcal{T}^+ = \{s \xrightarrow{a} s' \mid s, s' \in \mathcal{S}^+, a \in \mathcal{A}, \mathrm{pre}_a \subseteq s, s' = s[\![a]\!]^+\}$, where for all $a \in \mathcal{A}(s)$, $s[\![a]\!]^+ = s \cup \mathrm{eff}_a$.
- $\mathcal{S}^+_G = \{s \in \mathcal{S}^+ \mid G \subseteq s\}$.

$\rightsquigarrow$ Delete relaxation doesn't "delete" any fact when applying actions, i.e., facts are preserved from state to state. Thus, whenever $s \xrightarrow{a} s'$, we have that $s \subseteq s'$.

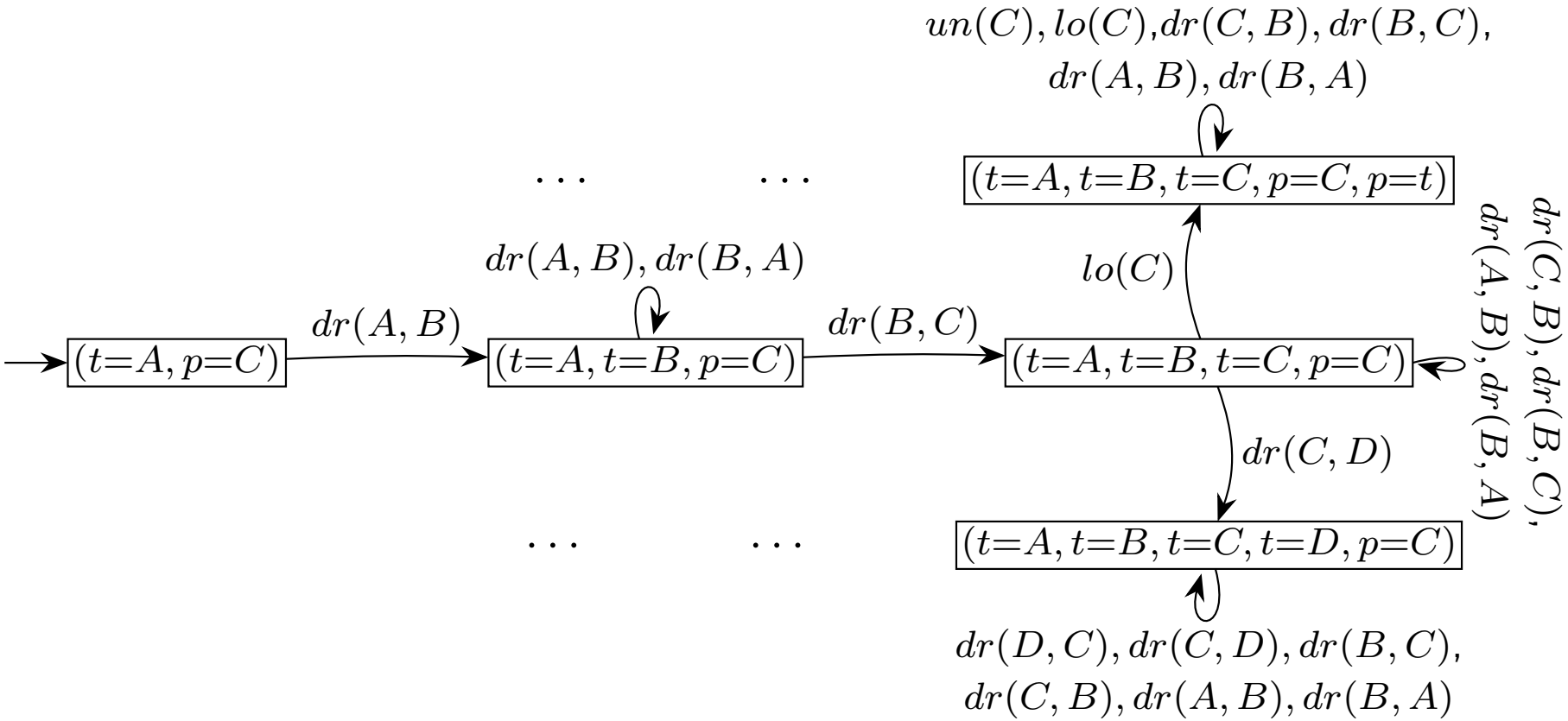
## Definition (Relaxed Plan)

Let $\Pi = (\mathcal{V}, \mathcal{A}, \mathrm{cost}, I, G)$ be an FDR planning task and $\Theta^+_\Pi$ its delete-relaxed state space. $\pi$ is a delete-relaxed (optimal) $s$-plan/plan for $\Pi$ if $\pi$ is (optimal) $s$-plan/plan in $\Theta^+_\Pi$.

# Delete Relaxation: Example




Very small fraction of the delete-relaxed state space:




A delete-relaxed plan: $(drive(A,B), drive(B,C), drive(C,D), lo(C), un(D))$

# Delete Relaxation Heuristics

**How can we use delete relaxation for computing heuristics?**

**Heuristic estimate = Cost of a delete-relaxed plan**

But before that, let's get few insights about delete-relaxed plans:

**Few observations:**

$\rightsquigarrow$ Whenever $s \xrightarrow{a} s'$, we have that $s \subseteq s'$:
States only grow as we apply more actions.

$\rightsquigarrow$ So, whenever $s \xrightarrow{a} s'$, we also have that $\mathrm{pre}_a \subseteq s'$:
Once an action $a$ is applicable it remains applicable in all subsequent states.

**Theorem (Every plan is a relaxed plan)**

Let $\Pi$ be an FDR planning task.
If $\pi$ is an $s$-plan for $\Pi$, then $\pi$ is a delete-relaxed $s$-plan for $\Pi$.

**Proof sketch.** Let $\pi = (a_1, \ldots, a_n)$, $s_0 = s$ and $s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \ldots \xrightarrow{a_n} s_n$.
Now, we can show by induction over $0 \leq i \leq n$ that applying $\pi$ on $s_0^+ = s_0$ in $\Theta_\Pi^+$ leads to $s_0^+ \xrightarrow{a_1} s_1^+ \xrightarrow{a_2} \ldots \xrightarrow{a_n} s_n^+$ such that $s_i \subseteq s_i^+$.

# Delete Relaxation Heuristics: Optimal delete-relaxed plans

**Definition ($h^+$)**

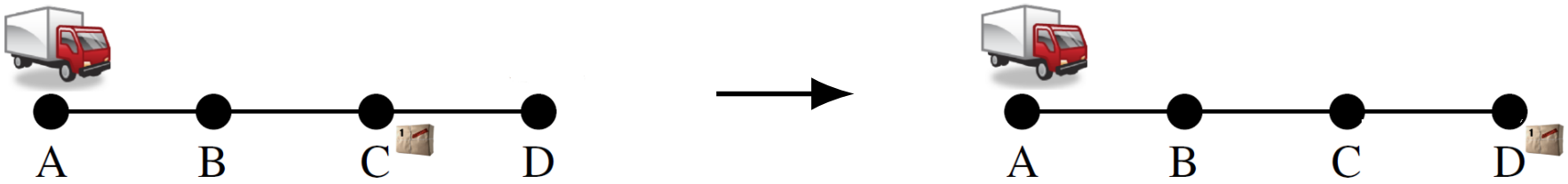
Let $\Pi$ be an FDR planning task.
$h^+$ is a heuristic for $\Pi$ that maps each state $s$ to the cost of optimal delete-relaxed $s$-plan, or to $\infty$ if no such $s$-plan exists.

**Theorem ($h^+ \leq h^*$)**

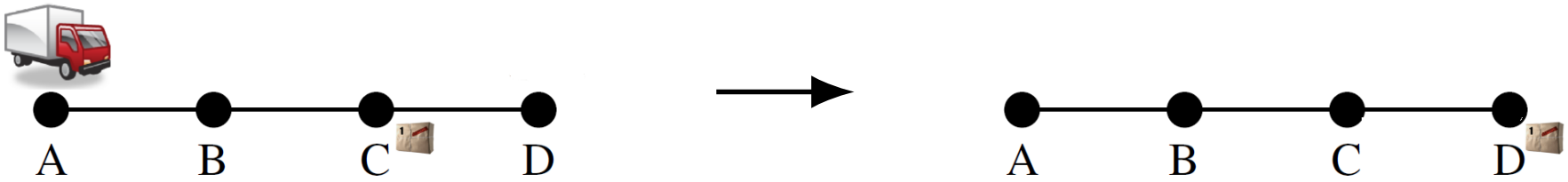
For every state $s$, $h^+(s) \leq h^*(s)$, and thus $h^+$ is an admissible heuristic for $\Pi$.

**Proof.** By contradiction: Let $\pi$ be an optimal $s$-plan, and let $\pi^+$ be an optimal delete-relaxed $s$-plan. Suppose $h^+(s) > h^*(s)$. Clearly, $h^*(s) = \mathrm{cost}(\pi)$ and $h^+(s) = \mathrm{cost}(\pi^+)$ therefore we have that $\mathrm{cost}(\pi^+) > \mathrm{cost}(\pi)$, but $\pi$ is also a delete-relaxed plan, therefore $\pi^+$ cannot be optimal.

# $h^+$: Example

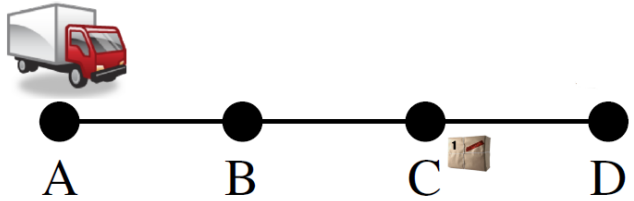



optimal plan:
$(dr(A,B), dr(B,C), lo(C), dr(C,D), un(D), dr(D,C), dr(C,B), dr(B,A))$
optimal delete-relaxed plan: $(dr(A,B), dr(B,C), lo(C), dr(C,D), un(D))$
$h^*(I) = 8, h^+(I) = 5$

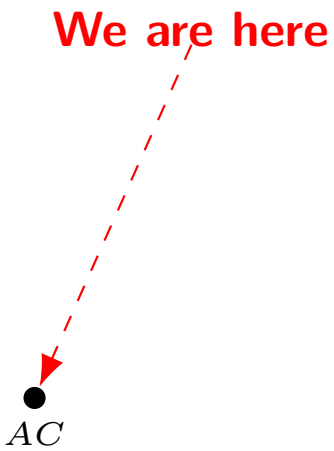



optimal plan: $(dr(A,B), dr(B,C), lo(C), dr(C,D), un(D))$
optimal delete-relaxed plan: $(dr(A,B), dr(B,C), dr(C,D), lo(C), un(D))$
$h^*(I) = 5, h^+(I) = 5$

# How to Relax During Search with $h^+$



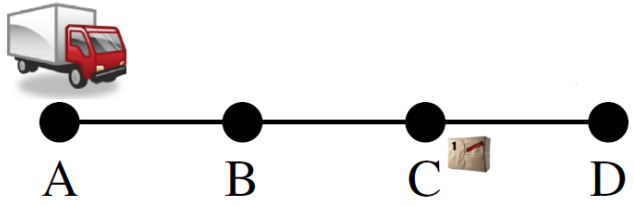

**Greedy best-first search:**
(tie-breaking: alphabetic)




**Real problem:**

- Initial state $I$: $AC$; goal $G$: $AD$.
- Action: $drXY, loX, ulX$.

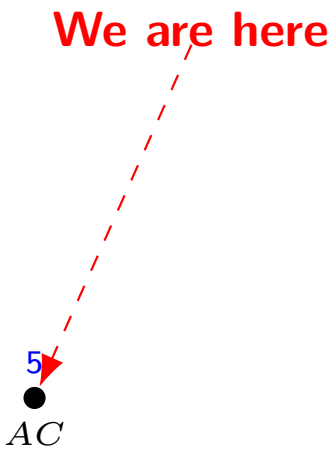
# How to Relax During Search with $h^+$



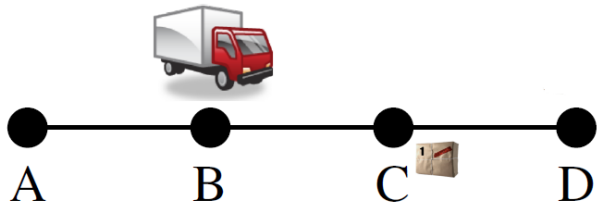

**Relaxed problem:**

- State $s$: $AC$; goal $G$: $AD$.
- $h^+(s) = 5$: e.g., $\langle drAB, drBC, drCD, loC, ulD \rangle$.

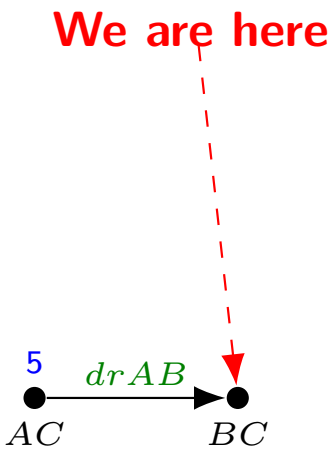
**Greedy best-first search:**
(tie-breaking: alphabetic)



# How to Relax During Search with $h^+$



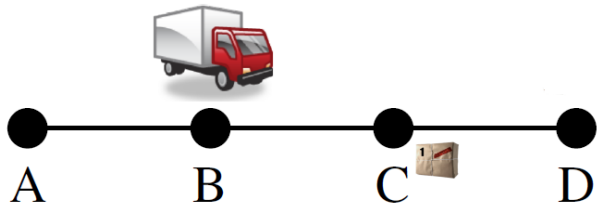

**Greedy best-first search:**
(tie-breaking: alphabetic)




**Real problem:**

- State $s$: $BC$; goal $G$: $AD$.
- $AC \xrightarrow{drAB} BC$.

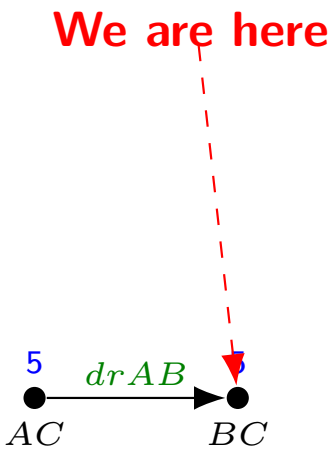
# How to Relax During Search with $h^+$



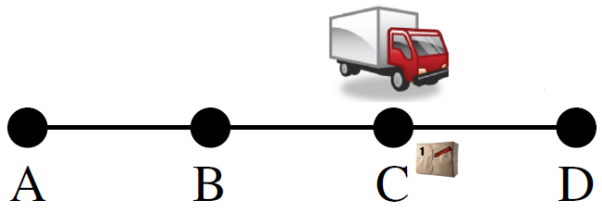

**Relaxed problem:**

- State $s$: $BC$; goal $G$: $AD$.
- $h^+(s) = 5$: e.g., $\langle drBA, drBC, drCD, loC, ulD \rangle$.

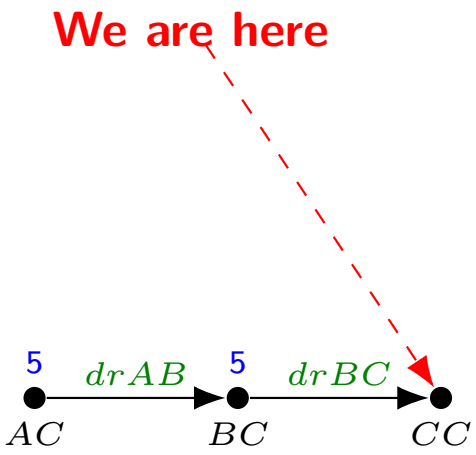
**Greedy best-first search:**
(tie-breaking: alphabetic)



# How to Relax During Search with $h^+$



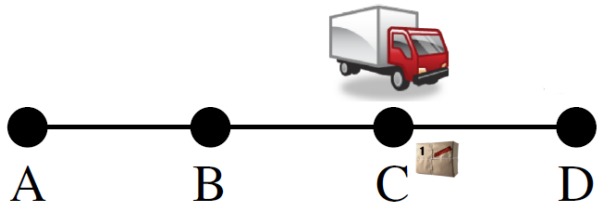

**Greedy best-first search:**
(tie-breaking: alphabetic)




**Real problem:**

- State $s$: $CC$; goal $G$: $AD$.
- $BC \xrightarrow{drBC} CC$.

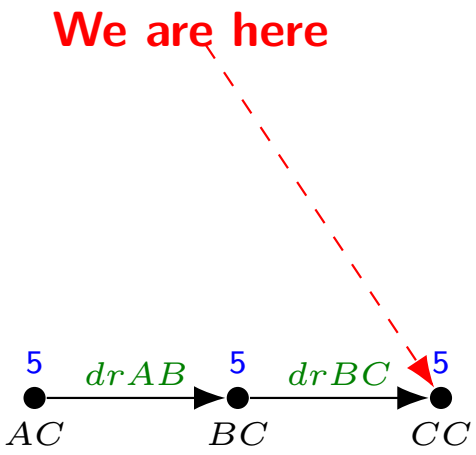
# How to Relax During Search with $h^+$



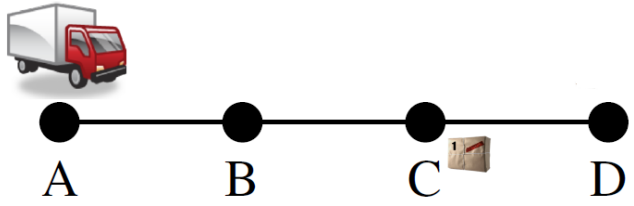

**Relaxed problem:**

- State $s$: $CC$; goal $G$: $AD$.
- $h^+(s)$ =5: e.g., $\langle drCB, drBA, drCD, loC, ulD \rangle$.

**Greedy best-first search:**
(tie-breaking: alphabetic)



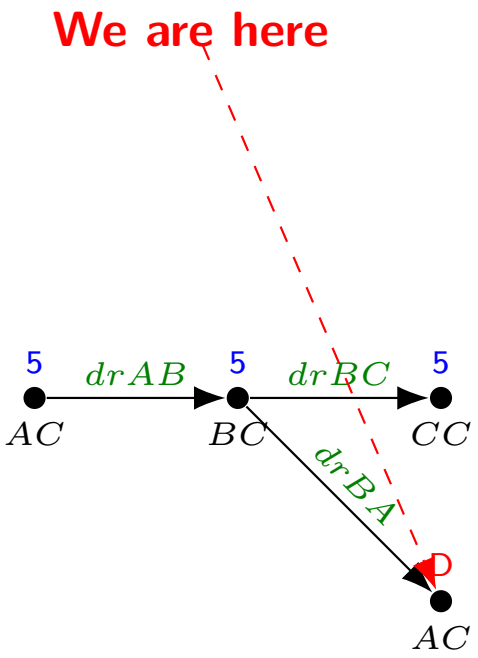
# How to Relax During Search with $h^+$



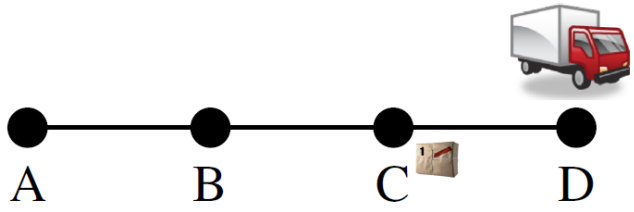

**Real problem:**

- State $s$: $AC$; goal $G$: $AD$.
- Duplicate state, prune.

**Greedy best-first search:**
(tie-breaking: alphabetic)



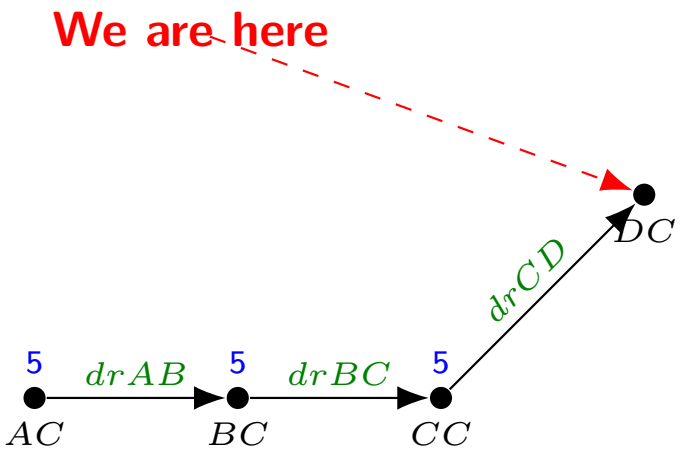
# How to Relax During Search with $h^+$



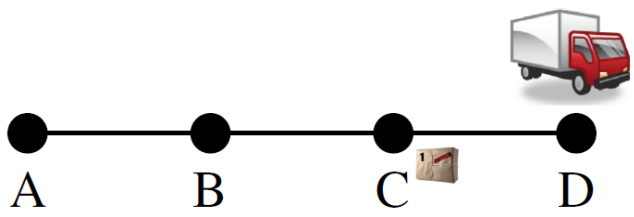

**Real problem:**

- State $s$: $DC$; goal $G$: $AD$.
- $CC \xrightarrow{drCD} DC$.

**Greedy best-first search:**
(tie-breaking: alphabetic)



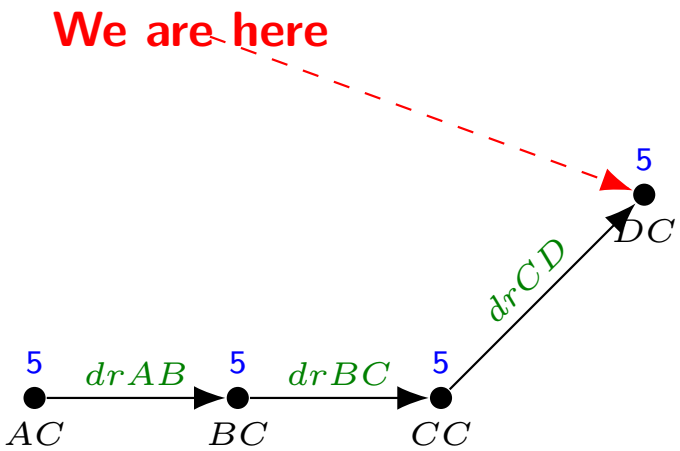
# How to Relax During Search with $h^+$



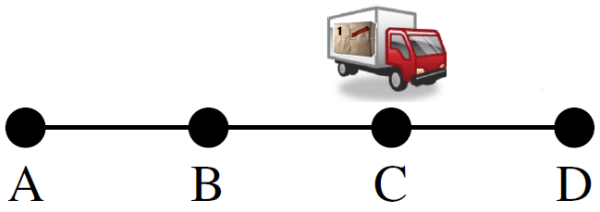

**Relaxed problem:**

- State $s$: $DC$; goal $G$: $AD$.
- $h^+(s) = 5$: e.g., $\langle drDC, drCB, drBA, loC, ulD \rangle$.

**Greedy best-first search:**
(tie-breaking: alphabetic)



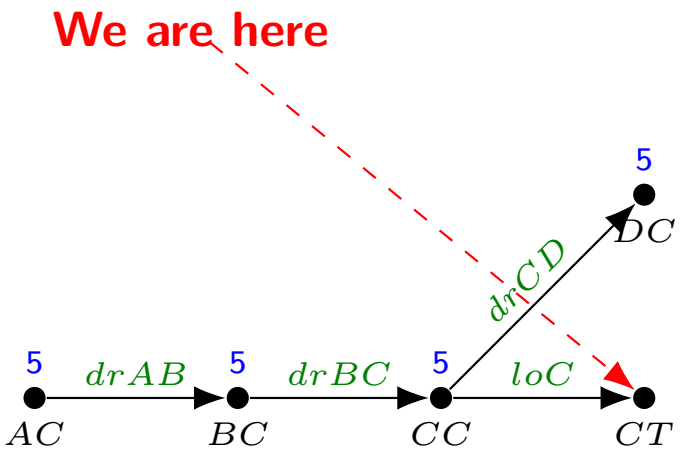
# How to Relax During Search with $h^+$



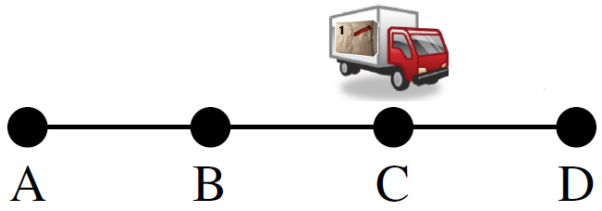

**Greedy best-first search:**
(tie-breaking: alphabetic)

**Real problem:**

- State $s$: $CT$; goal $G$: $AD$.
- $CC \xrightarrow{loC} CT$.



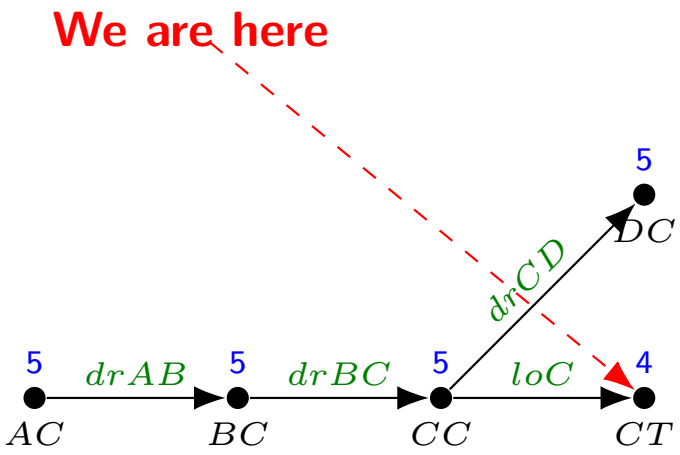
# How to Relax During Search with $h^+$



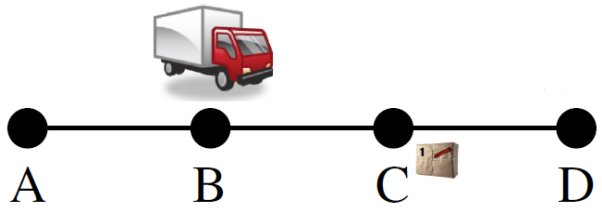

**Relaxed problem:**

- State $s$: $CT$; goal $G$: $AD$.
- $h^+(s) = 4$: e.g., $\langle drCB, drBA, drCD, ulD \rangle$.

**Greedy best-first search:**
(tie-breaking: alphabetic)



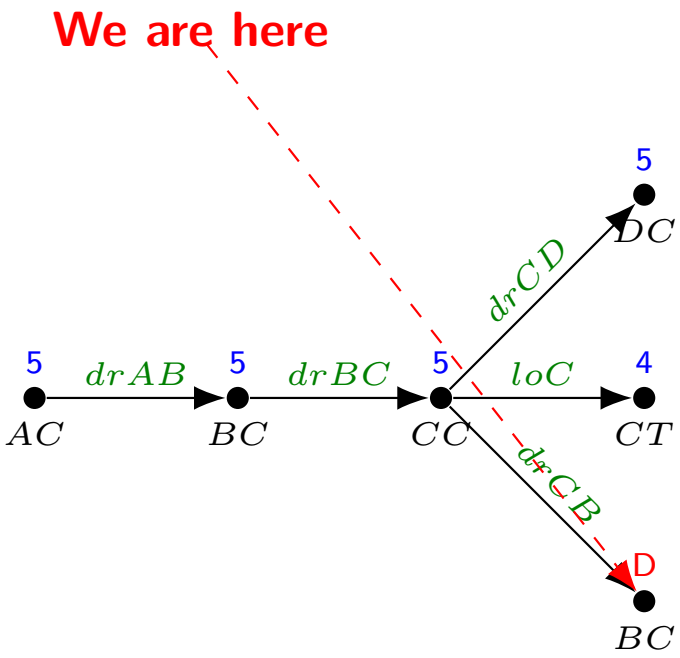
# How to Relax During Search with $h^+$



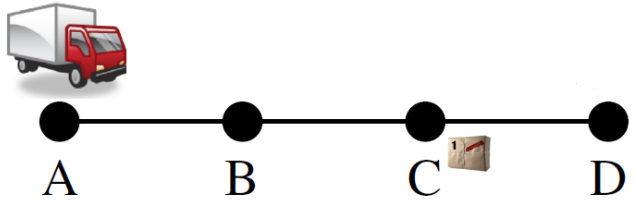

**Real problem:**

- State $s$: $BC$; goal $G$: $AD$.
- Duplicate state, prune.

**Greedy best-first search:**
(tie-breaking: alphabetic)



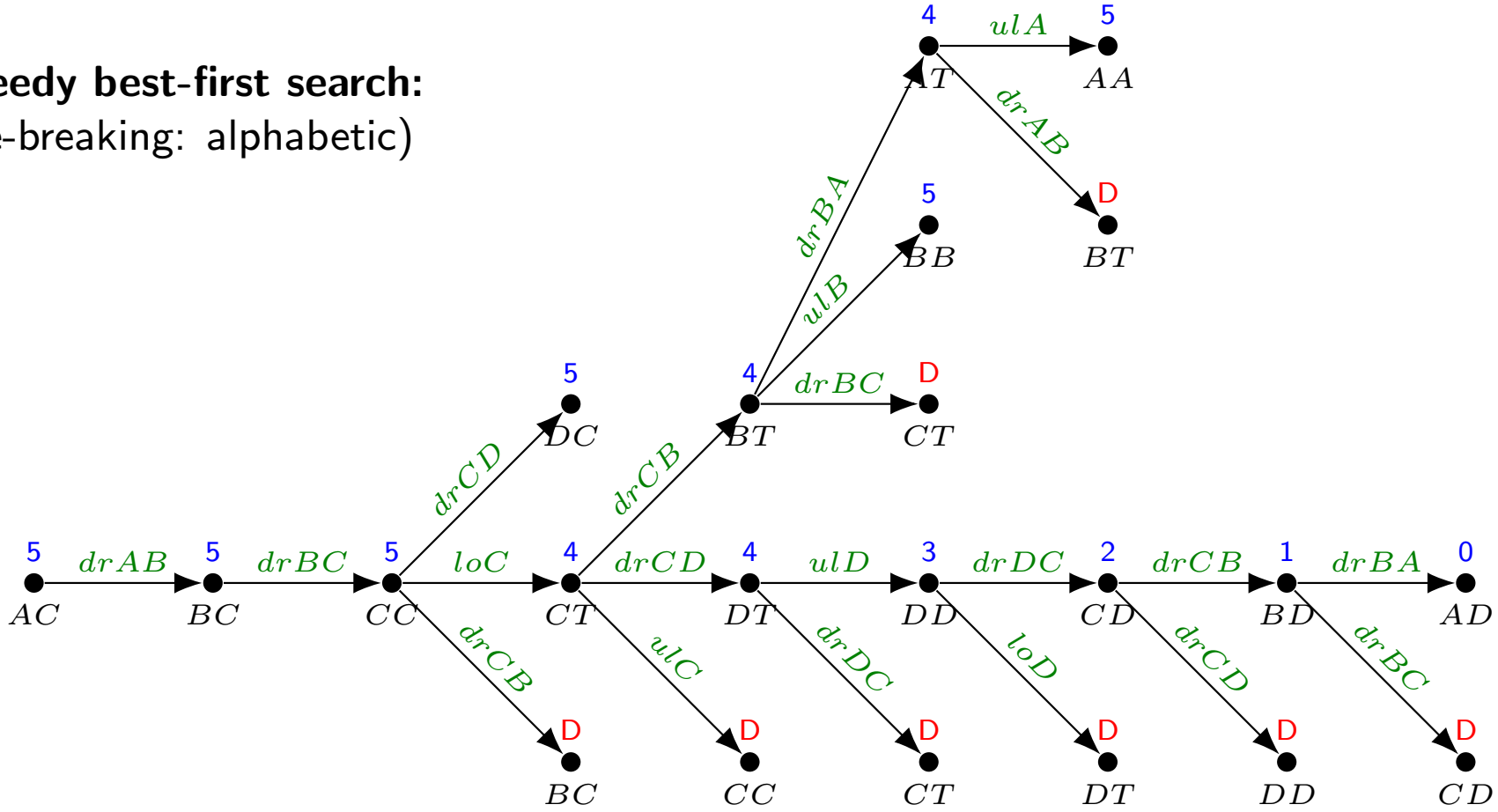
# How to Relax During Search with $h^+$



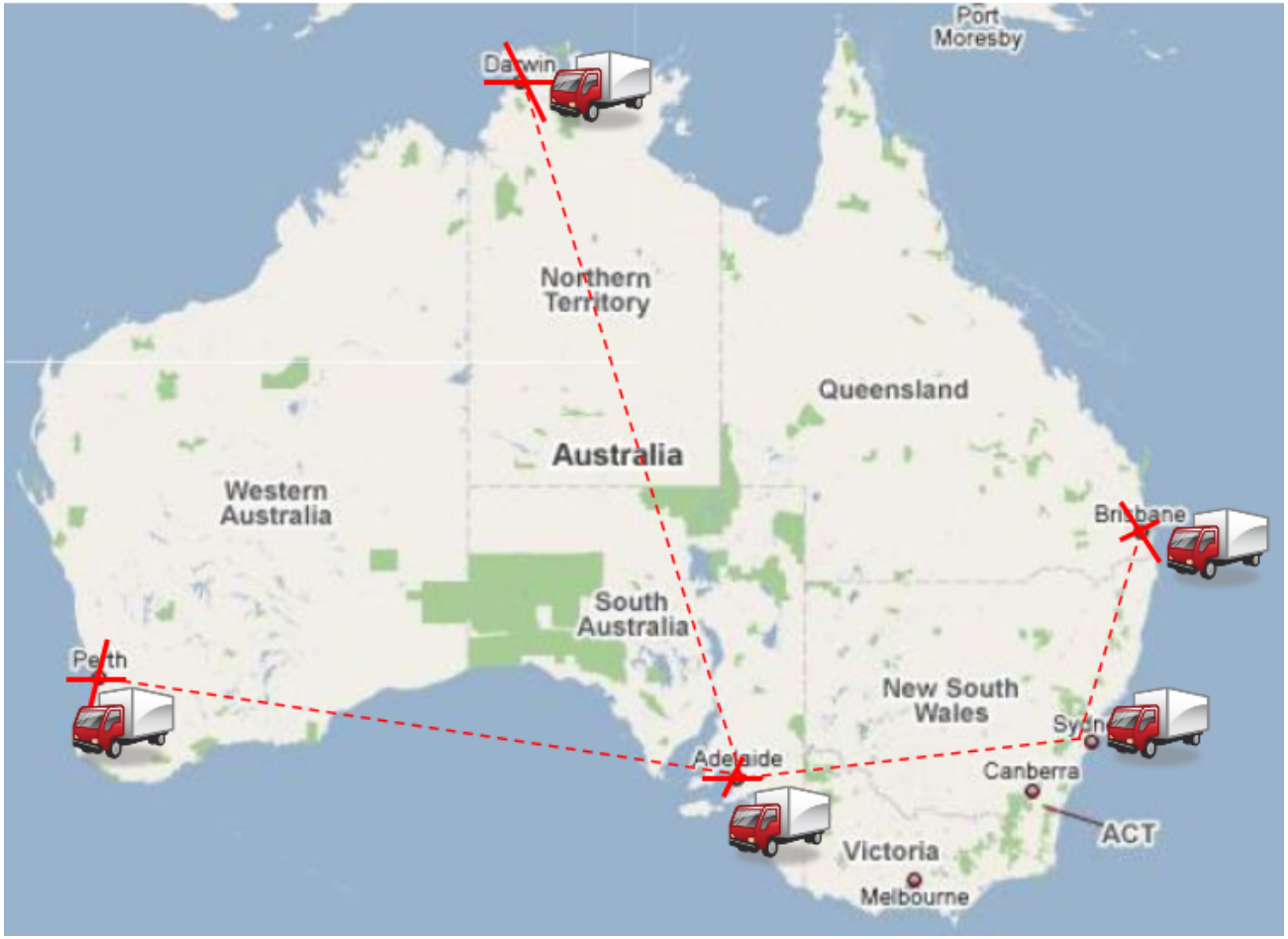

**Real problem:**

- Initial state $I$: $AC$; goal $G$: $AD$.
- Action: $drXY, loX, ulX$.

**Greedy best-first search:**
(tie-breaking: alphabetic)

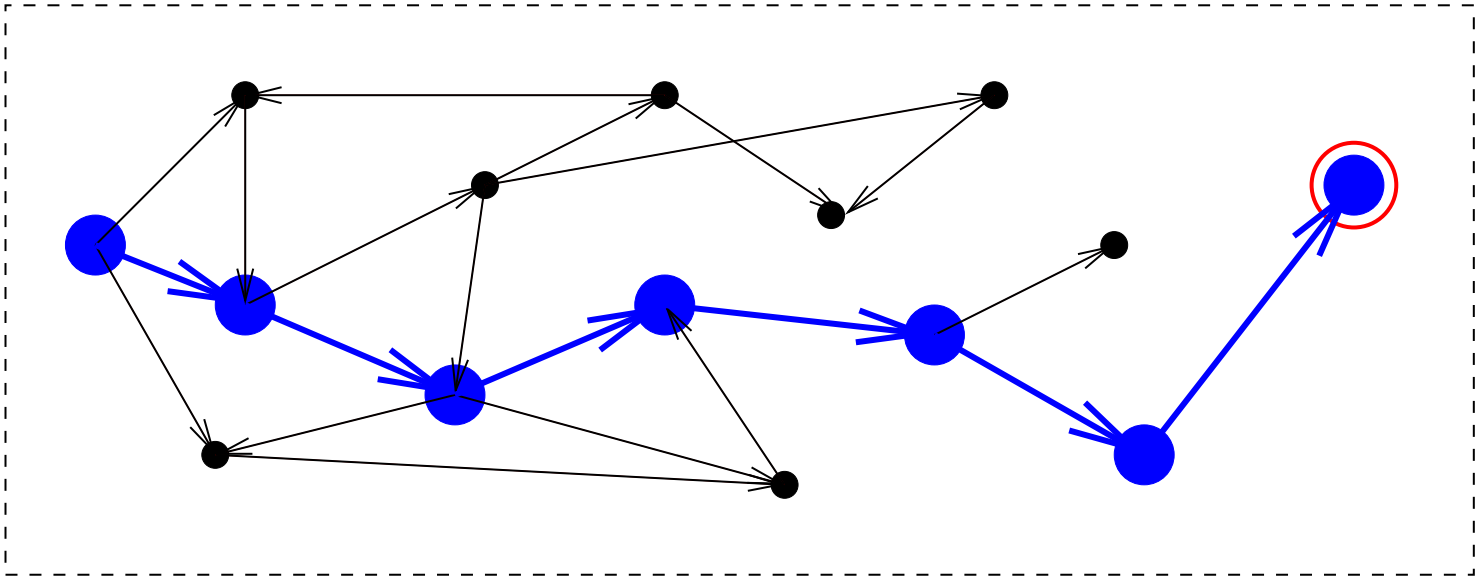


# A little bit of intuition: $h^+$ in (the Real) TSP




$\rightsquigarrow h^+ =$ **Minimum Spanning Tree**

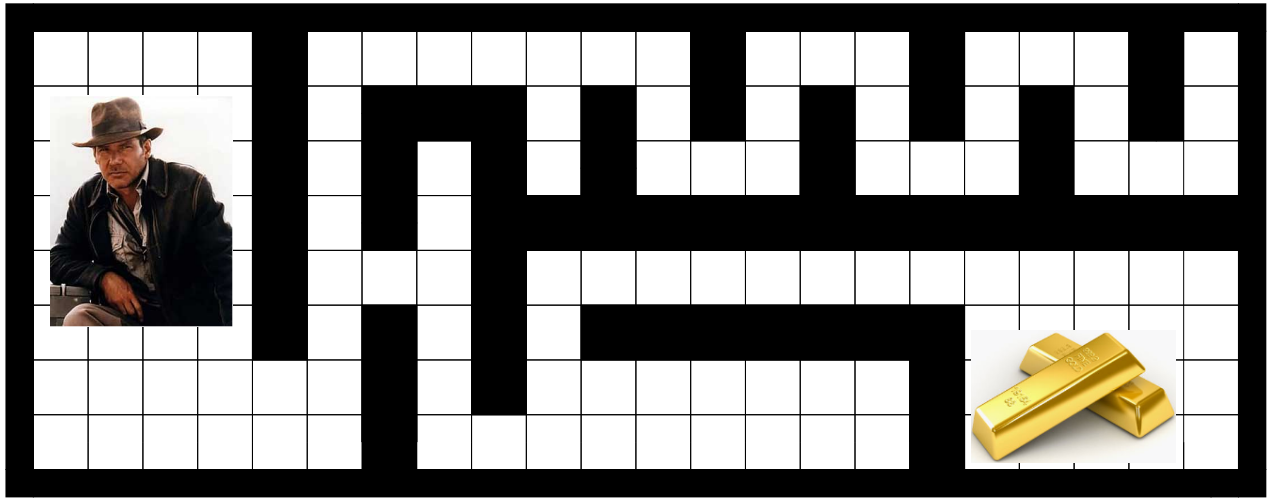
# A little bit of intuition: $h^+$ in Graphs



$h^+$(Graph-Distance) = real distance

(shortest paths never "walk back")

# Question



## Question

In this domain, $h^+$ is equal to?

(A): Manhattan Distance

(B): Horizontal distance

(C): Vertical distance

(D): $h^*$

$\rightsquigarrow$ (A): No, relaxed plans can't walk through walls. (B), (C): No, relaxed plans must move both horizontally and vertically. (D): Yes, optimal plan = shortest path = optimal relaxed plan (cf. previous slide).

# Computing Delete-Relaxation Heuristics

**Finally: How do we compute $h^+$?**

**Theorem (Optimal Delete-Relaxed Plan Existence)**

Given an FDR planning task $\Pi$ and an integer $K$, deciding whether there exists a delete-relaxed plan for $\Pi$ of length at most $K$ is NP-complete.

**There is no efficient (polynomial) algorithm for computing $h^+$!**

**But ...**

**Theorem (Delete-Relaxed Plan Existence)**

Given an FDR planning task $\Pi$, deciding whether there exists a delete-relaxed plan for $\Pi$ is in P.

So, we can compute **a** delete-relaxed plan, and use its cost as a (inadmissible) heuristic estimate.

# Computing Delete-Relaxation Heuristics

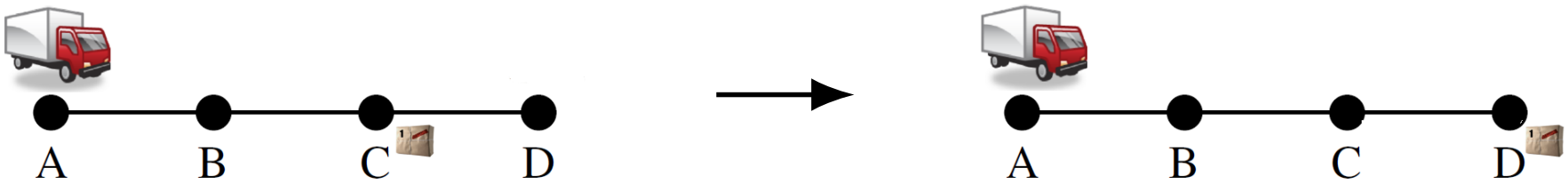
Let's try something simple first:

```
function SimpleDeleteRelaxationHeur(state s) returns a heuristic estimate
    F ← s /* Reached facts, initialized with the input state */
    C ← 0 /* Estimate of a delete-relaxed plan cost */
    A ← 𝒜 /* Actions not applied yet */
    while G ⊈ F do
        A′ ← {a ∈ A | pre_a ⊆ F} /* Remaining applicable actions */
        F′ ← F ∪ ⋃_{a∈A′} eff_a /* Build next layer of reached facts */
        if F = F′ then return ∞;
        C ← C + min_{a∈A′} cost(a) /* Update heuristic estimate */
        A ← A \ A′ /* Update the set of remaining actions */
        F ← F′ /* Switch to next layer */
    return C
```

$\rightsquigarrow$ Idea: Enumerate all facts reachable under delete-relaxation layer by layer, and for each step, account only for the cheapest action.

$\rightsquigarrow$ This heuristic will typically greatly underestimate $h^+$!

# Computing Delete-Relaxation Heuristics: Example

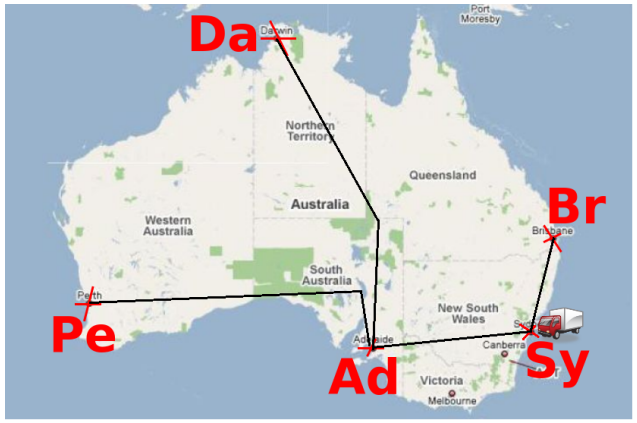



(For column $F$, only added facts are listed.)

| Step | $F$ | $C$ | $A'$ |
|---|---|---|---|
| 0. | $t = A, p = C$ | 0 | |
| 1. | $t = B$ | 1 | $drive(A, B)$ |
| 2. | $t = C$ | 2 | $drive(B, C), drive(B, A)$ |
| 3. | $t = D, p = t$ | 3 | $drive(C, D), drive(C, B), load(C)$ |
| 4. | $p = A, p = B, p = C, p = D$ | 4 | $drive(D, C), unload(A/B/C/D)$ |

Note: $h^+(s) = 5$

# Computing Delete-Relaxation Heuristics: Example




Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

| Step | $F$ | $C$ | $A'$ |
|---|---|---|---|
| 0. | $at = Sy, v(Sy)$ | 0 | |
| 1. | $at = Br, v(Br), at = Ad, v(Ad)$ | 1 | $drive(Sy, Br), drive(Sy, Ad)$ |
| 2. | $at = Pe, v(Pe), at = Da, v(Da)$ | 2 | $drive(Ad, Pe), drive(Ad, Da)$ |
| | | | $drive(Ad, Sy), drive(Br, Sy)$ |

Note: $h^+(s) = 10$

# Exercise Sheet

Exercise Sheet 3: Section 2

# Where's my Delete-relaxed Plan?

`SimpleDeleteRelaxationHeur` provides a heuristic estimate, but can we somehow obtain a delete-relaxed plan from it?

Yes, and here is a simple algorithm:

- Let $A_1, \ldots, A_n$ be the set of applicable actions $A'$ from 1st,...,nth cycle of the algorithm, respectively.
- Order actions in each $A_i$ arbitrarily and concatenate $A_1 \circ \ldots \circ A_n$, i.e., put actions from $A_1$ first in any order, then actions from $A_2$ in any order and so on.

The cost of such delete-relaxed plan will greatly overestimate $h^+$, because it includes all actions reachable under delete-relaxation.

**But we can do much better!** Here is how:

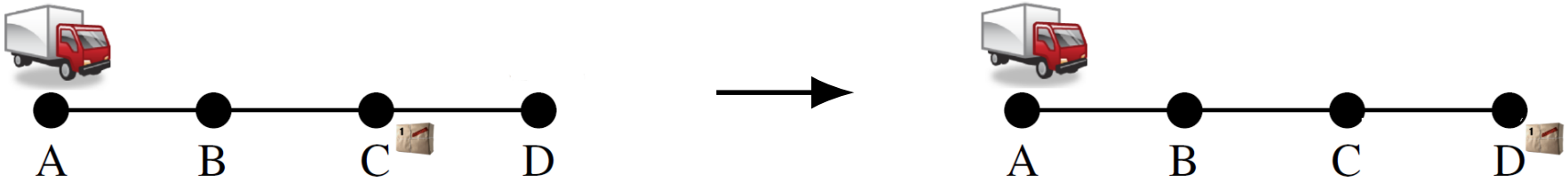
- Build layers of facts/actions reachable under delete relaxation as before.
- Chain backwards from goals and select only those actions that are needed to achive goals.

# FF Heuristic

```
function FF(state s) returns a heuristic estimate
    F_0 ← s /* First fact layer initialized with the input state */
    m ← 0 /* Counter of layers */
    while G ⊈ F_m do
        A_m ← {a ∈ 𝒜 | pre_a ⊆ F_m} /* Actions applicable in F_m */
        F_{m+1} ← F_m ∪ ⋃_{a∈A_m} eff_a /* Compute next fact layer */
        if F_m = F_{m+1} then return ∞;
        m ← m + 1;
    for each f ∈ F_m do level(f) = min{t | f ∈ F_t} /* First layer with f */;
    for each a ∈ A_{m−1} do level(a) = min{t | a ∈ A_t} /* First layer with a */;
    for t = 1, ..., m do G_t ← {g ∈ G | level(g) = t} /* (Sub)goals at layer t */ ;
    for t = m, ..., 1 do
        for each g ∈ G_t do
            Select any a ∈ 𝒜 such that level(a) = t − 1 and g ∈ eff_a;
            C ← C + cost(a);
            /* Add preconditions of a as subgoals */
            for each f ∈ pre_a do G_{level(f)} ← G_{level(f)} ∪ {f} ;
    return C
```

$\rightsquigarrow$ FF heuristic is **not** admissible, because it finds *some* delete-relaxed $s$-plan, necessarily cheaper than optimal (unrelaxed) $s$-plan.

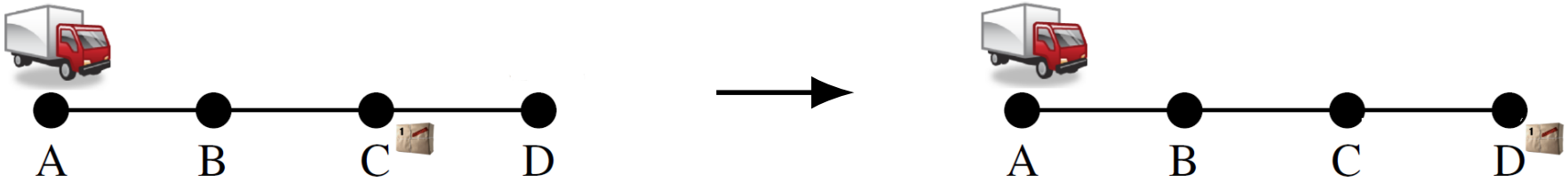
# FF: Example




- $F_0 = \{t = A, p = C\}$

**Build $F_i$, $A_i$ layers**

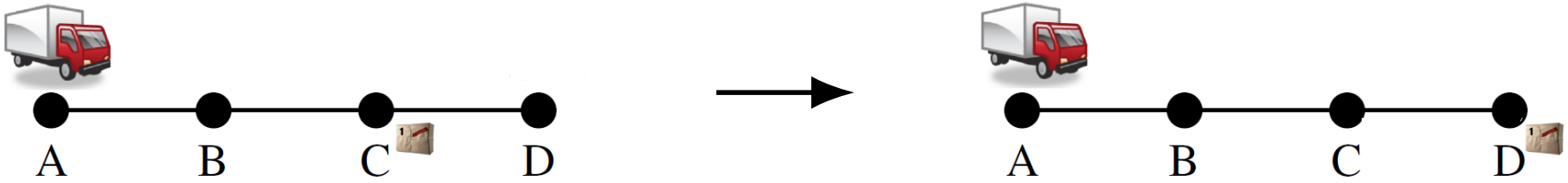
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$

**Build $F_i$, $A_i$ layers**

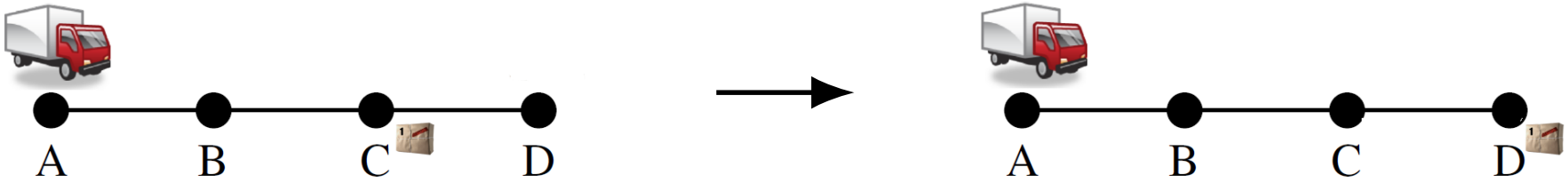
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$

**Build $F_i$, $A_i$ layers**

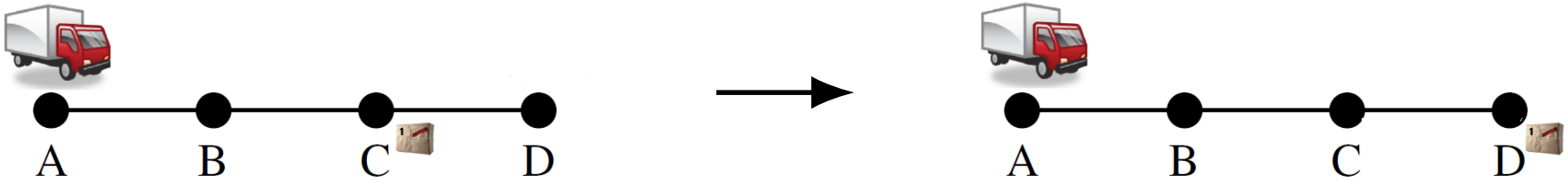
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$

**Build $F_i$, $A_i$ layers**

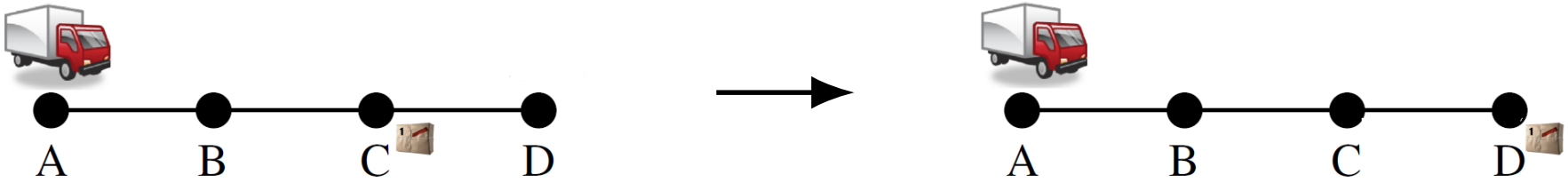
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$

**Build $F_i$, $A_i$ layers**

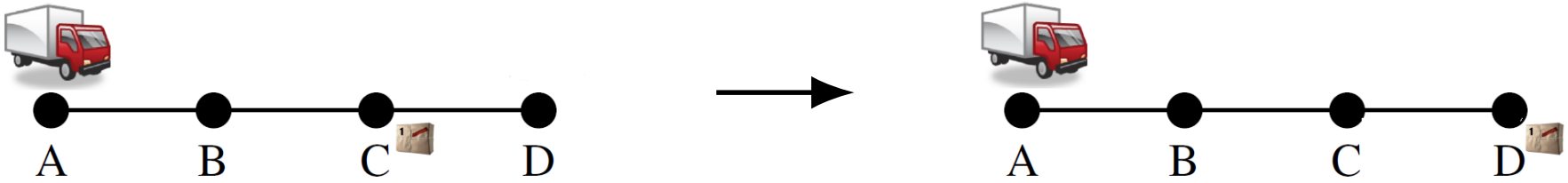
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$

**Build $F_i$, $A_i$ layers**

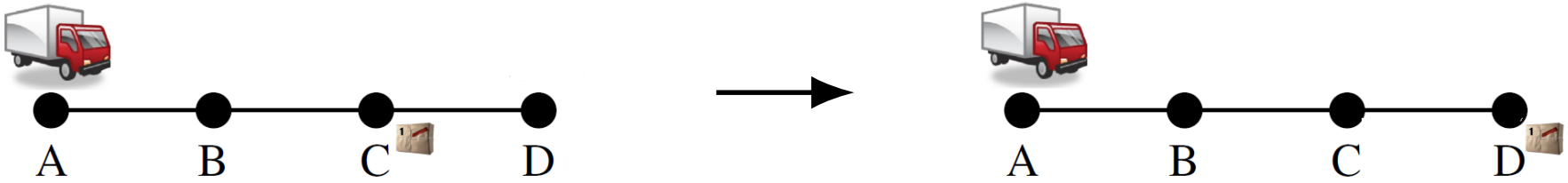
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$

**Build $F_i$, $A_i$ layers**

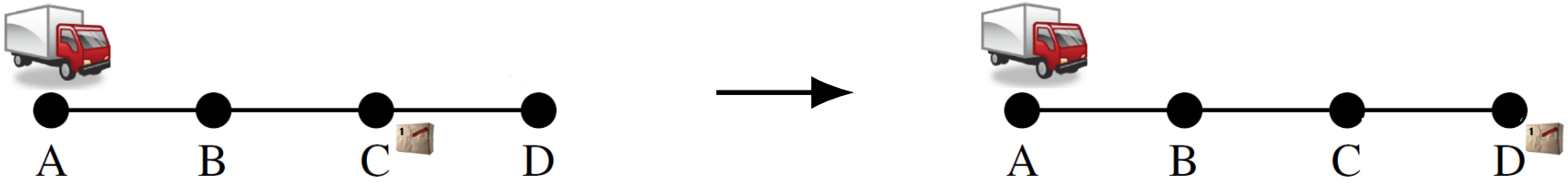
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$

**Build $F_i$, $A_i$ layers**

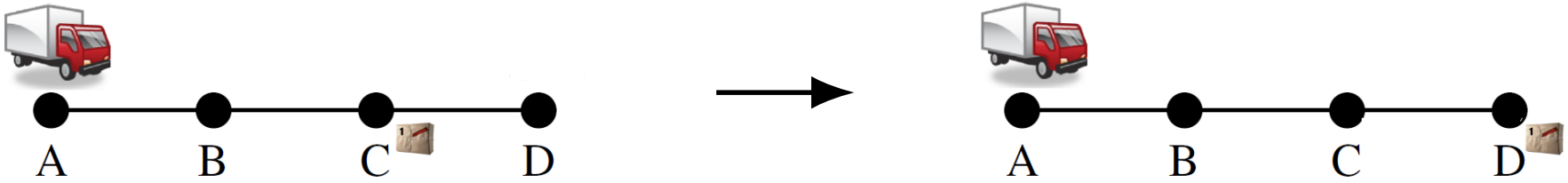
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Build $F_i$, $A_i$ layers**

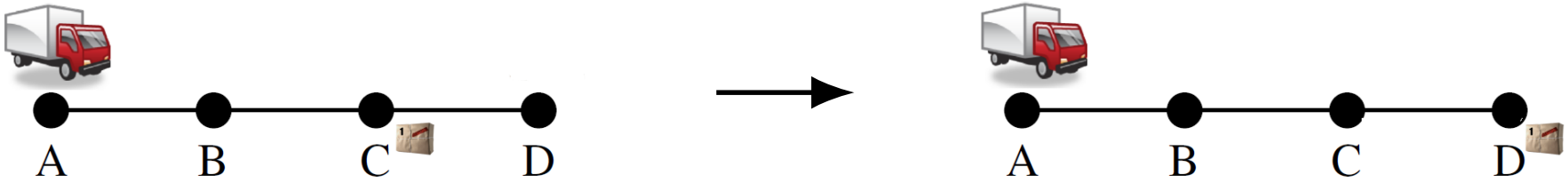
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Mark goals**

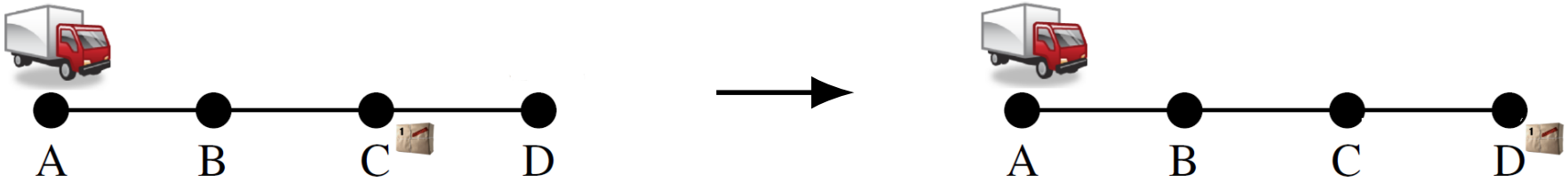
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Supporting subgoals at** $t = 4$

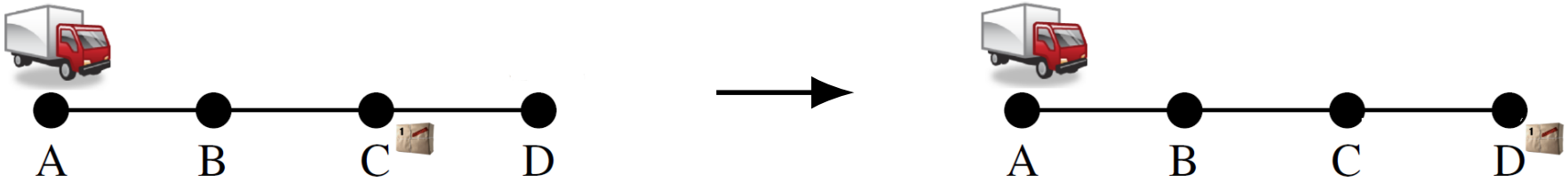
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Supporting subgoals at** $t = 4$

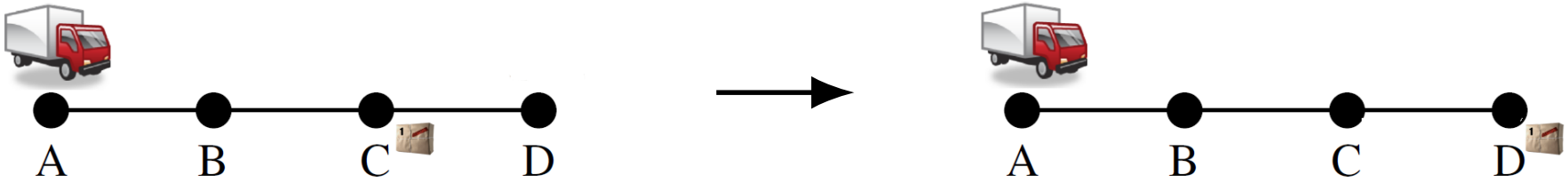
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Supporting subgoals at** $t = 3$

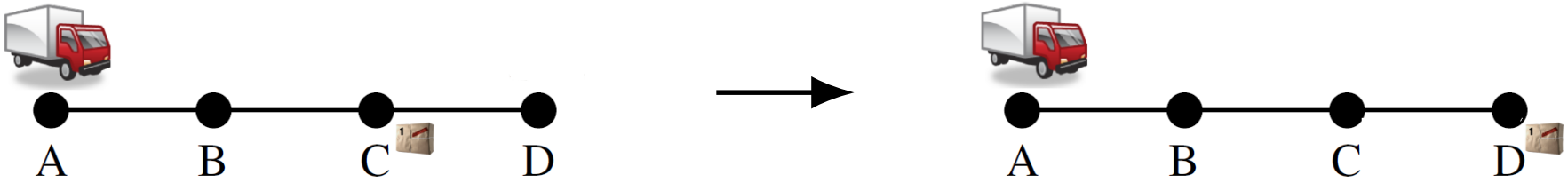
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Supporting subgoals at** $t = 3$

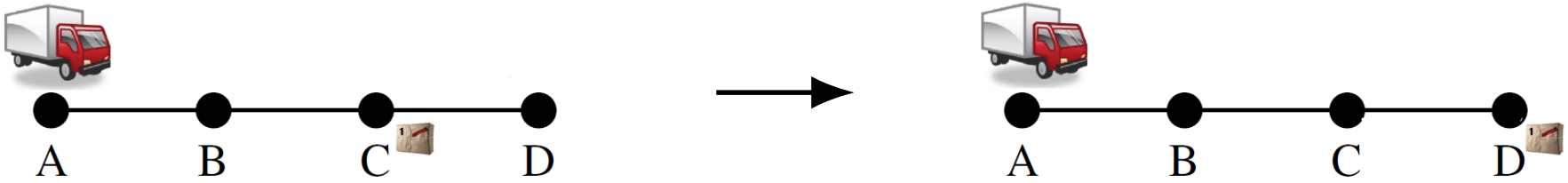
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Supporting subgoals at** $t = 2$

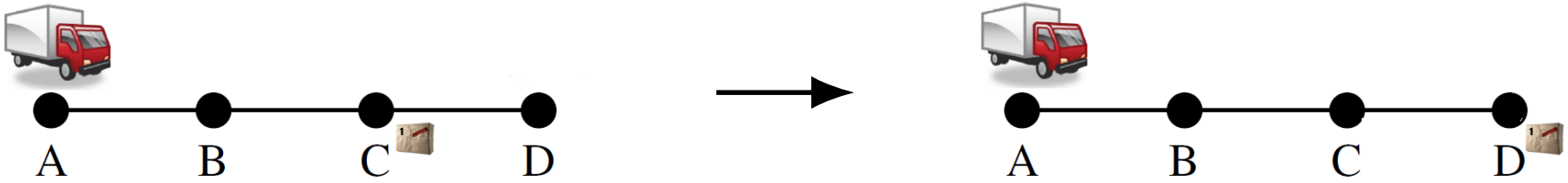
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

**Supporting subgoals at** $t = 1$

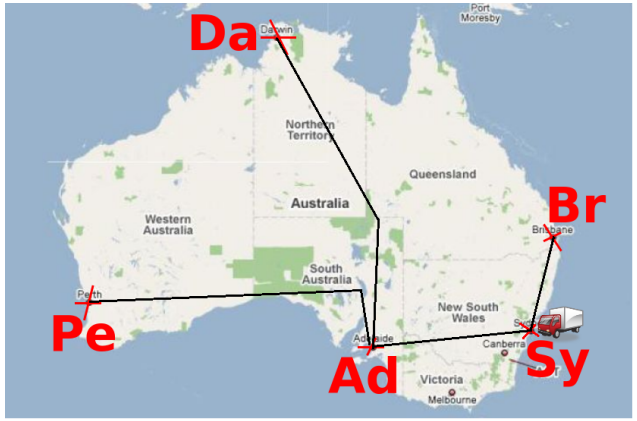
# FF: Example




- $F_0 = \{t = A, p = C\}$
- $A_0 = \{drive(A, B)\}$
- $F_1 = F_0 \cup \{t = B\}$
- $A_1 = \{drive(B, C), drive(B, A)\}$
- $F_2 = F_1 \cup \{t = C\}$
- $A_2 = \{drive(C, D), drive(C, B), load(C)\}$
- $F_3 = F_2 \cup \{t = D, p = t\}$
- $A_3 = \{drive(D, C), unload(A), unload(B), unloadC), unload(D)\}$
- $F_4 = F_3 \cup \{p = A, p = B, p = C, p = D\}$

$h^{FF}(s) = 5 = h^{+}(s)$

# FF: Example



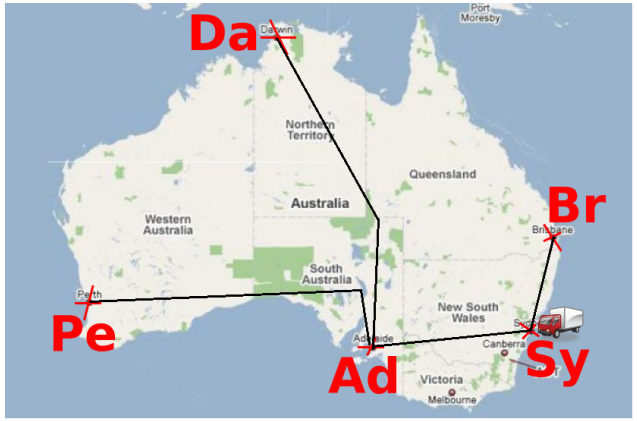

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$

**Build $F_i$, $A_i$ layers**

# FF: Example



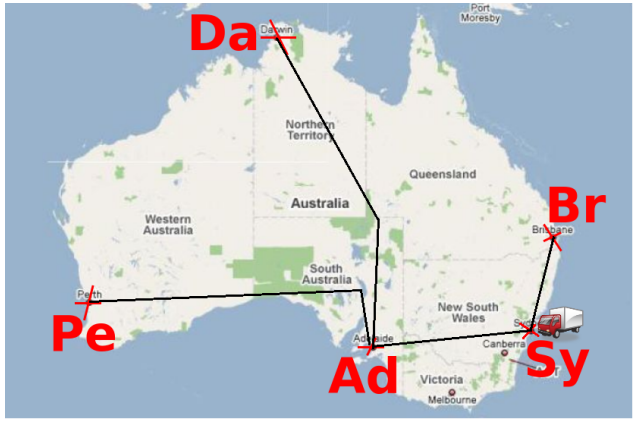

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$

**Build $F_i$, $A_i$ layers**

# FF: Example



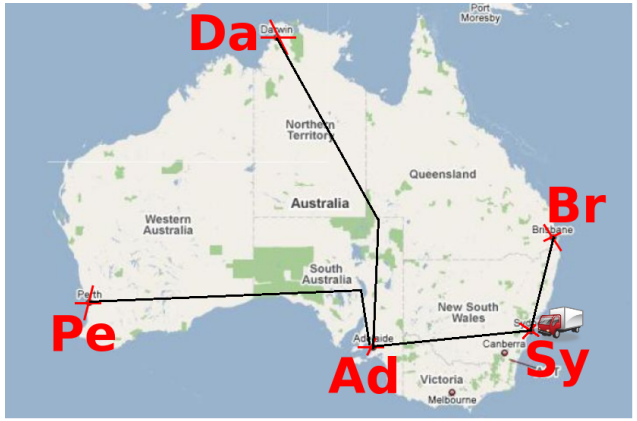

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$

**Build $F_i$, $A_i$ layers**

# FF: Example



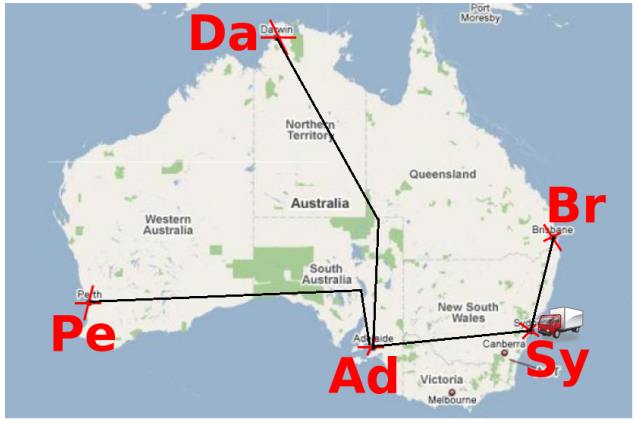

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$

**Build $F_i$, $A_i$ layers**

# FF: Example



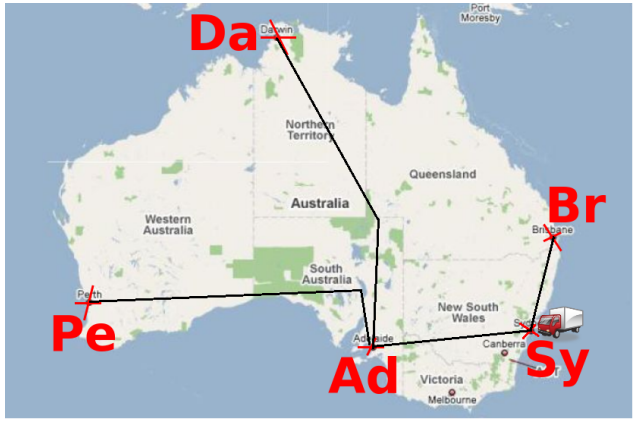

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

**Build $F_i$, $A_i$ layers**

# FF: Example



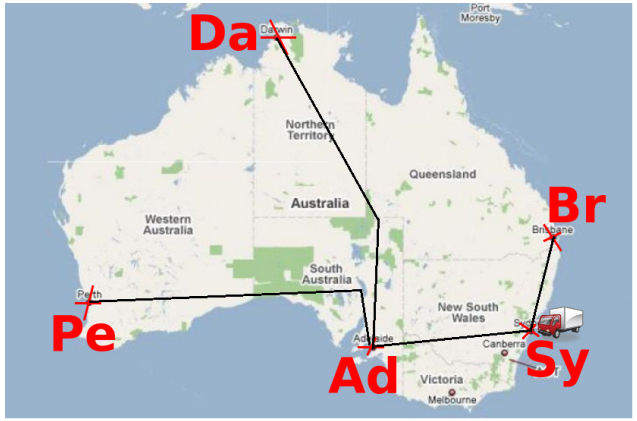

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

**Mark goals**

# FF: Example



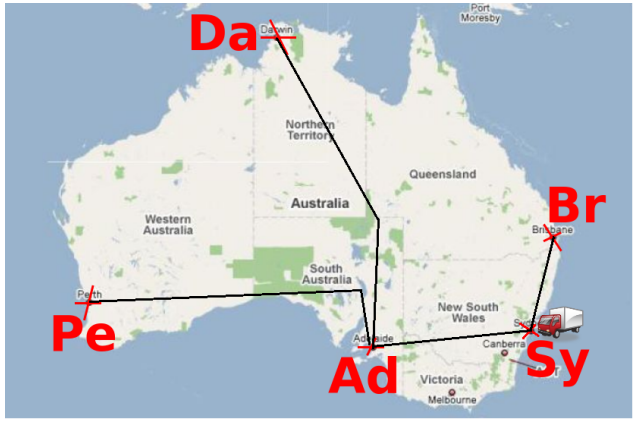

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

**Supporting subgoals at $t = 2$**

# FF: Example



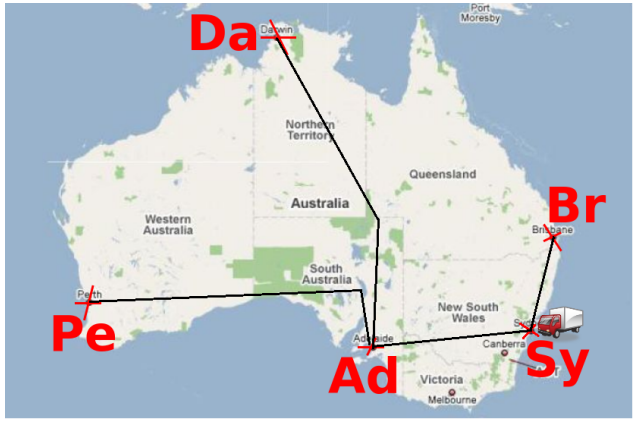

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

**Supporting subgoals at $t = 2$**

# FF: Example



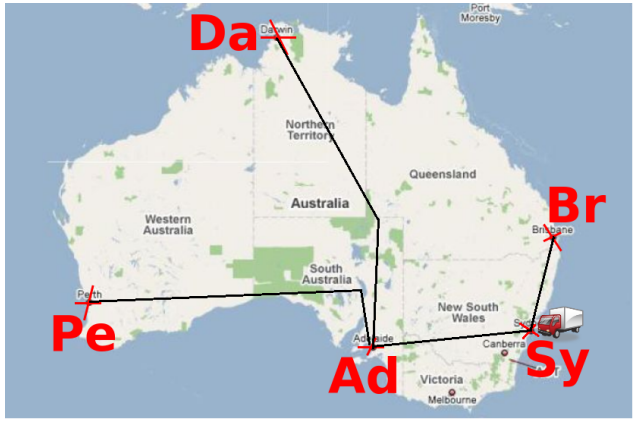

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

**Supporting subgoals at $t = 1$**

# FF: Example



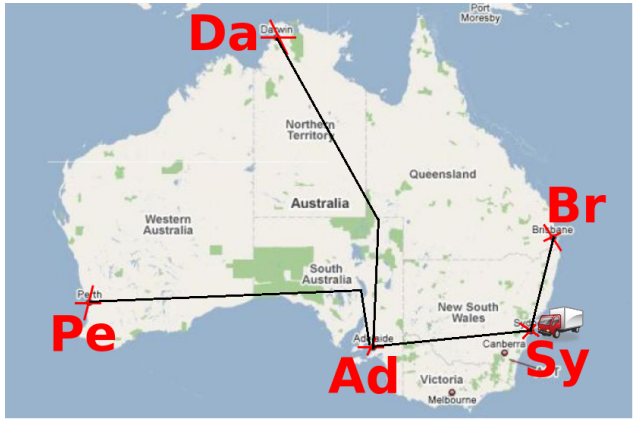

Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

$h^{FF}(s) = 10$

# FF: Example




Variables $\mathcal{V}$: $at : \{Sy, Ad, Br, Pe, Da\}$;
$v(x) : \{T, F\}$ for $x \in \{Sy, Ad, Br, Pe, Da\}$.
Initial state $I$: $at = Sy, v(Sy) = T, v(x) = F$ for $x \neq Sy$.
Goal $G$: $at = Sy, v(x) = T$ for all $x$.
Actions: $drive(x, y)$: pre: $at = x$, eff: $at = y, v(y) = T$.

$$\text{cost}(drive(x, y)) = \begin{cases} 1 & \{x, y\} = \{Sy, Br\} \\ 2 & \{x, y\} = \{Sy, Ad\} \\ 3 & \{x, y\} = \{Ad, Pe\} \\ 4 & \{x, y\} = \{Ad, Da\} \end{cases}$$

- $F_0 = \{at = Sy, v(Sy)\}$
- $A_0 = \{drive(Sy, Br), drive(Sy, Ad)\}$
- $F_1 = F_0 \cup \{at = Br, v(Br), at = Ad, v(Ad)\}$
- $A_1 = \{drive(Ad, Pe), drive(Ad, Da), drive(Ad, Sy), drive(Br, Sy)\}$
- $F_2 = F_1 \cup \{at = Pe, v(Pe), at = Da, v(Da)\}$

$h^{FF}(s) = 10 = h^+(s)$

# FF: Properties and Additional Notes

- FF heuristic is **not** admissible, because it finds some delete-relaxed $s$-plan, but not necessarily an optimal one.
- There are other ways to compute FF heuristics—we can make selections of actions in different ways.
- There are other delete relaxation heuristics, e.g.:
  - $h^{\max}$: Approximation of achieving the mostly costly subgoal $g \in G$.

$$h^{\max}(s,g) := \begin{cases} 0 & g \subseteq s \\ min_{a \in A, p \in \mathrm{eff}_a} 1 + h^{\max}(s, \mathrm{pre}_a) & g = \{p\} \\ \max_{p \in g} h^{\max}(s, \{p\}) & |g| > 1 \end{cases}$$

    $\rightsquigarrow$ Admissible, but very uninformative (under-estimates vastly).
  - $h^{\mathrm{add}}$: Sum over cost of subgoals.

$$h^{\mathrm{add}}(s,g) := \begin{cases} 0 & g \subseteq s \\ min_{a \in A, p \in \mathrm{eff}_a} 1 + h^{\mathrm{add}}(s, \mathrm{pre}_a) & g = \{p\} \\ \sum_{p \in g} h^{\mathrm{add}}(s, \{p\}) & |g| > 1 \end{cases}$$

    $\rightsquigarrow$ Not admissible, and prone to over-estimation; FF works better.
  - LM-Cut: Landmarks—sets of actions that must be hit by all plans.
    $\rightsquigarrow$ Good lower bound on $h^+$.

# Exercise Sheet

Exercise Sheet 3: Section 3